

PROJET ETSIA

« YANSNET – PARTIE APPLICATION »

DOCUMENTATION API

DATA, BACKEND & INTERFACE ADMINISTRATEUR

GROUPE PROJET 15

MAKONGNE DEFFO July Merveille
NANKAP NDIZEU Loic Aurel
NKOULOU Joseph Emmanuel
NOUNGOUA NOUNGOUA Teddy Steven
TOFA DEFFO Lionel Junior

Promotion X2027

ANNEE ACADEMIQUE 2024 – 2025

Sommaire

1. Introduction & Architecture Globale.....	3
2. API Data & Intelligence Artificielle.....	4
2.1 Architecture des données (27 tables PostgreSQL).....	4
2.2 Modèles de Machine Learning.....	5
2.3 Endpoints de l'API Data (Flask – port 5002).....	7
2.4 Sécurité & Conformité Data.....	8
3. API Backend Utilisateur (Node.js/Express – port 3000).....	9
3.1 Vue d'ensemble et flux de traitement.....	9
3.2 Authentification & Gestion des comptes	9
3.3 Gestion du contenu (posts, médias).....	10
3.4 Interactions sociales (likes, commentaires, follows).....	11
3.5 Messagerie privée & Groupes de discussion.....	12
3.6 Notifications & Signalements.....	12
3.7 Rate Limiting & Restrictions	13
4. Interface Administrateur – Site Web.....	13
4.1 Vue d'ensemble et accès.....	13
4.2 Gestion des utilisateurs	14
4.3 Modération du contenu.....	14
4.4 Tableau de bord & Statistiques	15
4.5 Configuration du système	15
5. Matrice de Contrôle d'Accès (RBAC).....	16
6. Flux de Données.....	17
7. Glossaire.....	18

1. Introduction & Architecture Globale

Ce document constitue la référence technique complète des API et interfaces du projet YANSNET. Il couvre les trois composantes serveur du système :

- API Data & IA (Flask, port 5002) : modèles de Machine Learning pour la modération, l'analyse sentimentale et les recommandations
- API Backend Utilisateur (Node.js/Express, port 3000) : gestion CRUD des données, authentification JWT, WebSocket temps réel
- Interface Administrateur (React/Vite, port 5173) : tableau de bord de supervision, modération manuelle et gestion des utilisateurs

Ces trois services interagissent selon l'architecture suivante : le frontend mobile et web communiquent exclusivement avec le Backend Node.js, qui délègue les tâches d'analyse IA au serveur Data Python. La base de données PostgreSQL (27 tables, 8 domaines fonctionnels) sert de couche de persistance centrale partagée.

Adresses par défaut en développement :

Backend Node.js : <http://localhost:3000>

API Data & IA Flask : <http://localhost:5002>

Interface Admin Web : <http://localhost:5173>

Base de données : PostgreSQL — localhost:5432

2. API Data & Intelligence Artificielle

2.1 Architecture des données (27 tables PostgreSQL)

L'architecture data de YANSNET est organisée autour de 27 tables PostgreSQL regroupées en 8 domaines fonctionnels. Les données sont stockées avec optimisation pour les requêtes temps réel (indexes, materialized views, cascading deletes). Les modèles ML sont interrogés via API Flask.

Domaine 1 — Authentification & Utilisateurs (5 tables)

- Users : compte principal, email @ucac-icam.com, rôle, avatar, bio, filière, promotion
- Roles, permissions, sessions : gestion RBAC granulaire

Domaine 2 — Contenu (6 tables)

- Posts : texte + médias, status (published/moderation/rejected), hashtags
- Comments, media, tags : contenu enrichi et indexation thématique

Domaine 3 — Interactions Sociales (4 tables)

- Likes, follows, notifications, messages_privés : interactions temps réel via WebSocket

Domaine 4 — Groupes & Communautés (3 tables)

- Groups, group_members, group_messages : discussions multi-utilisateurs, public ou privé

Domaine 5 — Modération & Conformité (4 tables)

- Reports, moderation_logs, user_suspensions, content_moderation, sensitive_content : audit complet des actions de modération

Domaine 6 — Machine Learning (4 tables)

- User_preferences, content_engagement, recommendation_logs, sentiment_analysis : historique et résultats des modèles IA

Domaine 7 — Audit & Compliance (1 table)

- Audit_logs : toutes les actions loggées avec user, timestamp, action et paramètres

Schéma de la table Users (extrait représentatif) :

id : UUID — PRIMARY KEY (uuid_generate_v4())
 username : VARCHAR(50) — UNIQUE NOT NULL, alphanumérique
 email : VARCHAR(100) — UNIQUE NOT NULL, format institutionnel
 password_hash : VARCHAR(255) — haché bcrypt (coût 10)
 full_name : VARCHAR(100) — NOT NULL
 role : VARCHAR(20) — CHECK : student | alumni | moderator | admin | concierge
 is_active : BOOLEAN — soft delete (jamais de suppression physique)
 created_at : TIMESTAMP — DEFAULT NOW()

2.2 Modèles de Machine Learning

YANSNET intègre 5 modèles de machine learning déployés via le serveur Flask. Chaque modèle est spécialisé et appelé à des moments précis du cycle de vie d'un contenu.

Modèle 1 — Analyse Sentimentale

Évalue l'état psychologique des utilisateurs à travers leurs publications pour détecter des signes de détresse, de surmenage ou de dépression.

- Modèle : distilbert-base-multilingual (fine-tuné français)
- 3 labels de sortie : POSITIVE, NEUTRAL, NEGATIVE
- Appelé lors de chaque publication d'un post
- Résultats stockés en table sentiment_analysis
- Agrégation hebdomadaire pour détection de tendance dépressive
- Seuil CRITIQUE : score de négativité > 0.85 sur 7 jours → alerte automatique aux concierges

Modèle 2 — Détection de Harcèlement

Identifie les commentaires harcelants, insultants ou discriminatoires avant leur insertion en base de données. Appelé de manière synchrone avant chaque publication de commentaire.

- Modèle : toxic-bert (fine-tuné français)
- Classification multi-label : insulte, discrimination, menace
- Seuil de rejet : 0.70 de confiance → HTTP 400 renvoyé au client
- Résultat : {harcelant: true, score: 0.89, type: 'insulte'}
- Action : commentaire bloqué + notification de raison à l'auteur

Modèle 3 — Détection de Spam

Identifie les posts spam, publicités automatisées ou comptes bots via l'analyse de patterns comportementaux.

- Détection de patterns : répétition de contenu, densité de liens, emojis excessifs
- Identification de bots par analyse de comportement temporel
- Seuil de blocage : 5 posts identiques en 10 minutes
- Action : quarantaine en file de modération manuelle

Modèle 4 — Censure Automatique (Images & Texte)

Service de modération visuelle et textuelle fonctionnant de manière indépendante (port 5001). Analyser tout contenu média avant publication.

- Détection NSFW/nudité : vision transformer Falconsai/nsfw_image_detection
- Détection de texte haineux : NLP toxic-bert
- Score de confiance par type de violation

- Résultats stockés en table sensitive_content pour audit
- Action : blocage immédiat ou mise en file de modération manuelle

Modèle 5 — Système de Recommandations

Recommande des amis et des posts pertinents basés sur le comportement utilisateur. Exécuté en batch toutes les 6 heures pour tous les utilisateurs actifs.

- Recommandation d'amis : même filière, amis en commun, interactions similaires
- Recommandation de posts : trending + personnalisation par préférences
- Algorithme : collaborative filtering basé sur content_engagement
- Freshness boost : posts récents favorisés dans le score de pertinence
- Stockage : table recommendation_logs

2.3 Endpoints de l'API Data (Flask – port 5002)

Le serveur Data expose les endpoints suivants, consommés exclusivement par le Backend Node.js. Toutes les requêtes et réponses sont en JSON.

GET /health — Vérifie l'opérationnalité du serveur et l'état de chargement des 4 modèles

Réponse : {"status":"ok","service":"YANSNET Data API","port":5002,"modeles":["sentiment","harcèlement","spam","recommandation"]}

POST /sentiment — Analyse le sentiment d'un texte unique

Entrée : {"texte": "je suis épuisé"}
Sortie : {"label":"NEGATIVE","score_pos":0.02,"score_neg":0.92,"en_detresse":true}

POST /sentiment/user — Analyse en batch les posts d'un utilisateur pour détection de détresse — appelé par le cron 6h

Entrée : {"user_id":"abc-123","posts":["texte1","texte2",...]}
Sortie : {"user_id":"abc-123","score_moyen_negatif":0.88,"alerte":true,"niveau":"CRITIQUE"}

POST /harcèlement — Analyse un commentaire avant insertion — HTTP 400 si score > 0.70

Entrée : {"texte": "tu es un imbécile"}
Sortie : {"harcelant":true,"score":0.89,"type":"insulte"}

GET /recommandations/:userId — Retourne les posts recommandés basés sur les préférences et engagements

Sortie : {"user_id":"abc-123","recommendations":[{"post_id":"xyz","score":0.92},...]}

2.4 Sécurité & Conformité Data

- Chiffrement E2E : messages privés chiffrés via Fernet (AES-128) — illisibles même en base de données
- Soft Delete : aucune suppression physique — droit à l'oubli via procédure SQL (is_active = false)
- Audit Log : toutes les actions enregistrées avec user, timestamp, action et paramètres
- Hachage : mots de passe en bcrypt (coût 10)
- JWT : tokens avec expiration 7 jours + refresh token pour renouvellement transparent
- Rate Limiting : max 10 posts/10 min, 50 DM/5 min, 30 commentaires/10 min par utilisateur
- SQL Injection : parameterized queries via ORM — aucune interpolation de chaînes

3. API Backend Utilisateur (Node.js/Express – port 3000)

3.1 Vue d'ensemble et flux de traitement

Le Backend est le point d'entrée unique de tous les clients (app mobile React Native, interface web administrateur). Programmé en Node.js/Express, il orchestre l'ensemble des opérations CRUD, délègue l'analyse IA au serveur Data, et distribue les événements en temps réel via WebSocket (Socket.io).

Flux de traitement d'une publication (exemple concret) :

- Étape 1 : le client envoie le contenu (texte + média) avec son token JWT
- Étape 2 : le Backend valide le JWT et extrait l'identité de l'utilisateur
- Étape 3 : le texte est envoyé à /harcelement et l'image à /censure (API Data)
- Étape 4a : si contenu valide → insertion en BD + notification WebSocket aux abonnés
- Étape 4b : si contenu douteux → file de modération manuelle (admin)
- Étape 5 : réponse retournée au client avec le statut de publication

3.2 Authentification & Gestion des comptes

Toutes les routes protégées exigent un token JWT valide dans l'en-tête Authorization. L'email doit obligatoirement être au format @ucac-icam.com.

POST /auth/register — Créer un compte utilisateur

- Corps requis : email (@ucac-icam.com), username, password (min 8 car., 1 majuscule, 1 chiffre), full_name
- Validations : unicité email + username, force du mot de passe, format institutionnel
- Retourne : {user, token} si succès — HTTP 400 si email dupliqué ou format invalide

POST /auth/login — Authentifier un utilisateur existant

- Corps requis : email + password
- Vérifie le hash bcrypt, génère un token JWT (exp 7j) + refresh token
- Retourne : {user: {id, username, email, role, avatar}, token, refreshToken}

POST /auth/refresh — Renouveler un token expiré sans re-saisie du mot de passe

GET /users/:id — Récupérer le profil public d'un utilisateur

PUT /users/:id — Modifier son profil (avatar, bio, filière, promotion, résidence) — token requis

GET /auth/check-email — Vérifier la disponibilité d'un email avant inscription

3.3 Gestion du contenu (posts, médias)

Toutes les routes de contenu nécessitent un token JWT valide. Avant insertion en base de données, chaque post passe par la chaîne de modération IA.

GET /posts — Récupérer le fil d'actualité personnalisé de l'utilisateur connecté

POST /posts — Créer un nouveau post (texte + média optionnel)

- Corps : {content: string, media?: File, hashtags?: string[]}
- Processus : validation JWT → analyse harcèlement → analyse NSFW (si image) → insertion → notification WebSocket
- Limites : max 50 Mo par image, max 20 Mo par vidéo, max 10 posts/10 minutes

GET /posts/:id — Récupérer un post par son identifiant avec ses statistiques

PUT /posts/:id — Modifier un post existant (auteur uniquement)

DELETE /posts/:id — Supprimer un post (auteur ou modérateur/admin)
 GET /posts/user/:userId — Récupérer tous les posts d'un utilisateur spécifique

3.4 Interactions sociales (likes, commentaires, follows)

POST /posts/:id/like — Liker un post — idempotent (un seul like par user/post)
 • Incrémente le compteur, envoie notification silencieuse à l'auteur, mise à jour temps réel

DELETE /posts/:id/like — Retirer son like — annule notification et décrémente le compteur

GET /posts/:id/comments — Lister tous les commentaires d'un post

POST /posts/:id/comments — Ajouter un commentaire — passe par l'analyse harcèlement

- Limites : max 30 commentaires/10 minutes par utilisateur
- Si harcèlement détecté (score > 0.70) : HTTP 400 + raison du rejet

PUT /comments/:id — Modifier son commentaire (auteur uniquement)

DELETE /comments/:id — Supprimer un commentaire (auteur ou modérateur)

POST /users/:id/follow — Suivre un utilisateur — ses posts apparaîtront dans le fil

DELETE /users/:id/follow — Ne plus suivre un utilisateur

GET /users/:id/followers — Lister les abonnés d'un utilisateur

GET /users/:id/following — Lister les abonnements d'un utilisateur

3.5 Messagerie privée & Groupes de discussion

Les messages privés sont chiffrés end-to-end via Fernet (AES-128) avant stockage. Les messages de groupe ne sont pas chiffrés car visibles par tous les membres.

GET /messages/:userId — Récupérer la conversation avec un utilisateur (messages déchiffrés à la volée)

POST /messages/:userId — Envoyer un message privé chiffré — distribué via WebSocket

- Corps : {content: string, file?: File (max 5 Mo)}

- Limite : max 50 messages par 5 minutes, impossible d'envoyer à quelqu'un qui vous a bloqué

GET /groups — Lister les groupes publics et ceux dont l'utilisateur est membre

POST /groups — Créer un groupe — l'auteur devient automatiquement admin du groupe

- Corps : {name, type: 'filierel'promotion'lrésidence'custom', description, isPrivate}

POST /groups/:id/join — Rejoindre un groupe public ou envoyer une demande (groupe privé)

POST /groups/:id/messages — Envoyer un message au groupe — distribué à tous les membres via WebSocket

GET /groups/:id/messages — Récupérer l'historique des messages d'un groupe

3.6 Notifications & Signalements

GET /notifications — Récupérer les notifications non lues de l'utilisateur connecté

PUT /notifications/:id/read — Marquer une notification comme lue

PUT /notifications/read-all — Marquer toutes les notifications comme lues

DELETE /notifications/:id — Supprimer une notification

Types de notifications générées automatiquement :

- LIKE : quelqu'un aime votre post (silencieuse — pas de son)
- COMMENT : quelqu'un commente votre post
- FOLLOW : quelqu'un commence à vous suivre
- MESSAGE : message privé reçu (bruyante — son activé)
- MENTION : vous êtes mentionné dans un commentaire
- GROUP_ADD : vous avez été ajouté à un groupe
- ALERT_SOS : alerte détresse (pour concierges uniquement)

POST /reports — Signaler un contenu inapproprié (anonyme vis-à-vis de l'auteur)

- Corps : {targetType: 'post'comment'user', targetId, reason: 'harcèlement'contenu_sexuel'violence'spam'faux_compte'autre'}
- Déclenche notification à tous les modérateurs actifs

- Abus de signalement (trop de signalements non fondés) → suspension temporaire

GET /reports — Lister les signalements en attente — MODERATOR/ADMIN uniquement

PUT /reports/:id — Traiter un signalement (approuver/rejeter) — MODERATOR/ADMIN uniquement

3.7 Rate Limiting & Restrictions

Limites appliquées par le middleware de rate limiting pour protéger les serveurs et prévenir les abus :

Créer des posts : Max 10 par 10 minutes
 Envoyer messages privés : Max 50 par 5 minutes
 Commenter : Max 30 par 10 minutes
 Liker : Max 100 par 10 minutes
 Créer des groupes : Max 5 par jour
 Upload de fichiers : Max 10 par heure
 Tentatives de connexion : Max 5 essais par 15 minutes
 Taille image : Max 3 Mo
 Taille vidéo : Max 20 Mo
 Fichier groupe : Max 50 Mo
 Vidéos par jour : Max 3 (économie bande passante)

4. Interface Administrateur – Site Web

4.1 Vue d'ensemble et accès

L'interface administrateur est une Single Page Application (SPA) développée avec React 18, Vite et TypeScript, stylisée avec shadcn/ui et TanStack Table. Elle est accessible exclusivement aux utilisateurs dont le rôle est admin ou moderator. Toute tentative d'accès sans token valide ou avec un rôle insuffisant est redirigée vers la page de connexion.

- URL par défaut : http://localhost:5173
- Authentification : token JWT identique à l'API Backend — même endpoint /auth/login

- Route protégée : ProtectedRoute vérifie isAuthenticated + isAdmin avant chaque rendu
- Déconnexion : invalidation du token local (localStorage) via AuthContext
- Variable d'environnement : VITE_API_URL (pointe vers http://localhost:3000 en dev)

4.2 Gestion des utilisateurs

Le module de gestion des utilisateurs permet aux administrateurs de superviser l'ensemble des comptes de la plateforme.

- Lister tous les utilisateurs avec filtres (rôle, statut actif/inactif, filière, promotion)
- Voir le profil complet d'un utilisateur (activité, posts, signalements reçus, suspensions)
- Activer ou désactiver un compte (soft delete — is_active = false)
- Changer le rôle d'un utilisateur : student → alumni → moderator → admin
- Suspendre temporairement un utilisateur avec durée et motif
- Consulter l'historique de modération d'un utilisateur

4.3 Modération du contenu

La file de modération centralise tous les contenus signalés ou mis en attente automatiquement par les modèles IA.

- File de modération : liste paginée des contenus en attente (posts, commentaires, images)
- Vue détaillée : contenu complet + score IA + raison du signalement + profil de l'auteur
- Actions disponibles : approuver (publier), rejeter (supprimer), suspendre l'auteur
- Signalements utilisateurs : voir les reports en attente, approuver ou rejeter
- Logs de modération : historique complet des décisions avec timestamp et modérateur
- Suppression directe : supprimer un post ou commentaire depuis l'interface sans passer par la file

4.4 Tableau de bord & Statistiques

Le tableau de bord offre une vue synthétique de l'activité de la plateforme en temps réel.

- KPIs principaux : nombre total d'utilisateurs actifs, posts publiés aujourd'hui, signalements en attente, alertes SOS non traitées
- Graphiques d'activité : évolution des inscriptions, publications et interactions (par jour/semaine/mois)
- Alertes SOS en cours : liste des étudiants en détresse détectés par le modèle sentimentale avec score critique
- Analytics de modération : taux de rejet automatique vs manuel, types de violations les plus fréquents
- Statistiques par rôle : répartition des utilisateurs par filière, promotion et rôle

4.5 Configuration du système

Les paramètres globaux de modération et d'alertes sont configurables depuis l'interface admin sans redémarrage du serveur.

- Seuils de modération IA : modifier les valeurs de confiance minimum pour harcèlement (défaut 0.70), NSFW (défaut 0.75), spam
- Alertes SOS : configurer le seuil de score de négativité critique (défaut 0.85 sur 7 jours)
- Rate limiting : ajuster les quotas par action (admin uniquement)
- Paramètres RBAC : activer/désactiver des permissions par rôle
- Audit log : consulter le journal complet des actions administrateur avec filtres

5. Matrice de Contrôle d'Accès (RBAC)

YANSNET implémente un système RBAC avec 5 rôles hiérarchiques. Chaque rôle hérite des permissions du rôle précédent et ajoute des capacités supplémentaires.

STUDENT (Étudiant) — rôle de base

- Créer, modifier et supprimer ses propres posts et commentaires
- Liker, commenter, suivre d'autres utilisateurs
- Envoyer des messages privés chiffrés E2E
- Créer et rejoindre des groupes de discussion
- Signaler du contenu inapproprié (anonymement)
- Voir ses notifications personnelles

ALUMNI (Ancien étudiant)

- Toutes les permissions STUDENT
- Accès aux archives des événements campus
- Engagement dans les groupes alumni

MODERATOR (Modérateur)

- Toutes les permissions ALUMNI
- Voir et supprimer tous les posts et commentaires de la plateforme
- Approuver ou rejeter les signalements (reports)
- Accéder à la file de modération (contenu suspect mis en attente par l'IA)
- Suspendre temporairement un utilisateur
- Accès aux messages privés interdit sauf sur signalement formel

ADMIN (Administrateur)

- Toutes les permissions MODERATOR
- Créer des publications institutionnelles
- Modifier les paramètres globaux de modération et les seuils IA
- Gérer les comptes utilisateurs (activation, désactivation, rôle)
- Accéder à l'audit log complet de toutes les actions
- Configurer les alertes SOS et les thresholds des modèles
- Voir les statistiques complètes de la plateforme

CONCIERGE (Responsable de résidence)

- Recevoir les alertes SOS des étudiants (score critique détecté par l'IA sentimentale)
- Voir les analytics simples (activité globale, tendances)
- Accès aux données personnelles interdit — anonymisation stricte

6. Flux de Données

Tableau des flux principaux de données dans YANSNET, avec entrées, traitements et sorties pour chaque fonctionnalité clé :

Inscription d'un utilisateur

Entrée : Email @ucac-icam.com, username, password, full_name

Traitement : Validation format → vérification unicité → hachage bcrypt → insertion BD

Sortie : Compte créé + token JWT

Publication d'un post

Entrée : Texte + média optionnel

Traitement : Analyse harcèlement (API Data) → censure image (port 5001) → insertion BD → WebSocket

Sortie : Post visible dans le fil des abonnés en temps réel

Envoi d'un commentaire

Entrée : Texte + post_id

Traitement : Analyse harcèlement synchrone → si OK : insertion + notification auteur

Sortie : Commentaire publié ou HTTP 400 (rejet IA)

Message privé

Entrée : Texte + fichier optionnel

Traitement : Chiffrement Fernet → stockage → Socket.io

Sortie : Message reçu en temps réel (chiffré en BD)

Analyse sentimentale (batch 6h)

Entrée : Tous les posts des 7 derniers jours par utilisateur

Traitement : API /sentiment/user → agrégation score → comparaison seuil 0.85

Sortie : Alerte SOS aux concierges si score CRITIQUE

Signalement de contenu

Entrée : targetType, targetId, reason

Traitement : Enregistrement → notification tous les modérateurs → file d'attente

Sortie : Post masqué en attente ou approuvé/rejeté par modérateur

7. Glossaire

UUID : Universally Unique Identifier — clé primaire 128 bits générée automatiquement

RBAC : Role-Based Access Control — contrôle d'accès par rôles avec permissions granulaires

JWT : JSON Web Token — jeton d'authentification signé (durée 7 jours), envoyé dans l'en-tête Authorization

E2EE : End-to-End Encryption — chiffrement de bout en bout des messages privés via Fernet/AES-128

NLP : Natural Language Processing — traitement automatique du langage naturel par les modèles IA

NSFW : Not Safe For Work — contenu visuel inapproprié détecté par vision transformer

SPA : Single Page Application — architecture frontend React où le routage est côté client

WebSocket : Protocole de communication bidirectionnel temps réel (Socket.io) pour les notifications et messages

Soft Delete : Suppression logique (is_active = false) — les données sont conservées pour audit

Rate Limiting : Limitation du débit de requêtes par utilisateur pour prévenir le spam et protéger les serveurs

Batch 6h : Traitement périodique automatique (toutes les 6 heures) pour l'analyse sentimentale et les recommandations

SOS Alert : Alerte de détresse psychologique déclenchée par score de négativité critique (> 0.85 sur 7 jours)

Fernet : Algorithme de chiffrement symétrique (AES-128-CBC + HMAC) utilisé pour les messages privés

distilbert : Modèle Transformer compact fine-tuné pour l'analyse sentimentale en français

toxic-bert : Modèle Transformer fine-tuné pour la détection de harcèlement et de contenu haineux

CRUD : Create, Read, Update, Delete — opérations de base sur les données

ORM : Object-Relational Mapping — couche d'abstraction évitant l'injection SQL

bcrypt : Algorithme de hachage adaptatif pour les mots de passe (coût 10 = ~100ms de calcul)